Reproducibility in Practice

Week 9: CS 203 - Software Tools and Techniques for AI

Prof. Nipun Batra

IIT Gandhinagar

The Friday Night Nightmare

11:30 PM — You (on WhatsApp):

> *"Done! Model hit 94% accuracy. Just pushed* `model_final_v2.py`*. We are going to ace this."*

11:45 PM — Your partner:

> *"I pulled it. I'm only getting 81%."*

11:47 PM — You:

> *"Wait, let me run it again... I'm getting 86% now?! What hyperparameters did I use an hour ago?"*

11:55 PM — Your partner:

> *"I tried to fix it. Now I get* `ModuleNotFoundError: No module named 'xgboost'`*. I'm on a Mac, are you on Windows?"*

12:30 AM: 17 Stack Overflow tabs. Still broken. **0 marks.**

Three Levels of Reproducibility

Every reproducibility failure comes from losing control of one of three things:

Level	Problem	Solution
1. The Math	"I get different results every run"	Seeds & determinism
2. The Memory	"Which of my 50 runs was the good one?"	Experiment tracking (TrackIO)
3. The Machine	"It works on my laptop but not yours"	Docker

Today we fix all three — **in order**.

Level 1: Control the Math

Seeds & Determinism

"Yesterday 92%, Today 85%, Same Code?!"

Run your model training twice — exact same code:

```
X_train, X_test, y_train, y_test = train_test_split(X, y)
model = RandomForestClassifier()
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

Run 1: 0.92 **Run 2:** 0.85 **Run 3:** 0.88

Which result do you report? The problem: **uncontrolled randomness.**

Sources of randomness in ML:

- **Train/test split** — which samples go where
- **Model initialization** — starting weights (neural nets)
- **Data shuffling** — order during training
- **Dropout** — which neurons to deactivate

The Minecraft Seed

In Minecraft, the world is randomly generated. But enter the **same seed** → get the **exact same world** every time.

Minecraft	Machine Learning
Seed number	`random_state=42`
Same seed → same world	Same seed → same split, same model
Share seed with friend → same map	Share seed with TA → same results

A random seed locks the universe into one specific timeline.

How to Lock the Universe

In sklearn: set `random_state` everywhere

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=100, random_state=42)

kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

Global seed (for larger projects):

```
import random, numpy as np

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)

set_seed(42)  # call once at the top of your script
```

Seeds: The Limits

Setting seeds makes YOUR code deterministic. But results can still differ across machines because of:

- **Different OS** → different BLAS/LAPACK math libraries under the hood
- **Different library versions** → sklearn 1.2 vs 1.4 may split data differently
- **GPU non-determinism** → CUDA operations are not always reproducible

Seeds solve **Level 1** (the math). We still need to solve the memory and the machine.

Level 2: Control the Memory

Experiment Tracking with TrackIO

The Spreadsheet Graveyard

You've been tuning hyperparameters all week:

```
run_lr001_depth5_est100.py     → 87.2%
run_lr01_depth10_est200.py     → 91.4%  ← wait, was this the one?
run_lr01_depth10_est200_v2.py  → 89.1%  ← or this?
run_FINAL.py                   → ???
run_FINAL_FINAL.py             → ???
run_FINAL_USE_THIS.py          → definitely not this
```

You had **the** best model last Tuesday. Now you can't find it.

Seeds made results reproducible. But you didn't record what you did.

TrackIO: Three Calls Is All You Need

TrackIO (by Hugging Face) — free, local-first experiment tracking.

```python
import trackio

# 1. Start a run
trackio.init(project="cs203-week08-demo", name="RandomForest",
             config={"model": "RF", "n_estimators": 100, "max_depth": 10})

# 2. Log metrics
trackio.log({"train_accuracy": 0.95, "test_accuracy": 0.845})

# 3. Finish
trackio.finish()
```

Everything stored locally in SQLite — no account, no cloud, no cost.

Demo: `python trackio_1_training_curves.py`

TrackIO: Training Curves

Log metrics at each step → dashboard shows curves:

```
trackio.init(project="cs203-week08-demo", name="gb-training",
             config={"model": "GradientBoosting", "lr": 0.1})

for n_est in range(10, 310, 10):
    gb = GradientBoostingClassifier(n_estimators=n_est, learning_rate=0.1,
                                    max_depth=3, random_state=42)
    gb.fit(X_train, y_train)
    trackio.log({
        "n_estimators": n_est,
        "train_accuracy": float(round(gb.score(X_train, y_train), 4)),
        "test_accuracy": float(round(gb.score(X_test, y_test), 4)),
    })

trackio.finish()
```

Watch accuracy climb and plateau — just like TensorBoard, but simpler.

TrackIO: Comparing Runs

Run the same model with 3 learning rates → dashboard **overlays** them:

```
for lr in [0.01, 0.1, 0.5]:
    trackio.init(project="cs203-week08-demo", name=f"lr-{lr}",
                 config={"learning_rate": lr})

    for n_est in range(10, 210, 10):
        gb = GradientBoostingClassifier(n_estimators=n_est, learning_rate=lr,
                                        random_state=42)
        gb.fit(X_train, y_train)
        trackio.log({
            "n_estimators": n_est,
            "train_accuracy": float(round(gb.score(X_train, y_train), 4)),
            "test_accuracy": float(round(gb.score(X_test, y_test), 4)),
        })

    trackio.finish()
```

Demo: `python trackio_3_compare_hyperparams.py`

TrackIO: What Did My Model Get Wrong?

Log **images** to see predictions, not just numbers:

```
trackio.init(project="cs203-week08-demo", name="prediction-analysis",
             config={"model": "RandomForest"})

# Create matplotlib grid of misclassified digits
fig = plot_misclassified_grid(X_test, y_test, preds)
fig.savefig("errors.png")

trackio.log({
    "misclassified": trackio.Image("errors.png",
        caption="All misclassified digits — red = wrong"),
    "test_accuracy": 0.97,
})
trackio.finish()
```

Demo: `python trackio_2_misclassified.py` → check "Media & Tables" tab

TrackIO: Per-Class Breakdown Table

Log **tables** for structured analysis:

```
trackio.log({"per_class_metrics": trackio.Table(
    dataframe=pd.DataFrame(
        table_data,
        columns=["Digit", "Precision", "Recall", "F1", "Support"]),
)})
```

Digit	Precision	Recall	F1	Support
0	1.000	0.970	0.985	33
5	0.938	0.957	0.947	23
9	0.950	0.950	0.950	20

Demo: `python trackio_4_per_class_table.py`

TrackIO: Alerts for Overfitting

Get notified when something goes wrong:

```
trackio.init(project="cs203-week08-demo", name="overfitting-detector",
             config={"model": "DecisionTree"})

for depth in range(1, 30):
    dt = DecisionTreeClassifier(max_depth=depth, random_state=42)
    dt.fit(X_train, y_train)
    gap = dt.score(X_train, y_train) - dt.score(X_test, y_test)

    trackio.log({"max_depth": depth, "overfit_gap": gap})

    if gap > 0.08:
        trackio.alert(title="Overfitting detected!",
                      text=f"Gap={gap:.1%} at depth={depth}",
                      level=trackio.AlertLevel.ERROR)
```

Demo: `python trackio_5_overfitting_alert.py`

TrackIO: The Dashboard

```
pip install trackio
trackio show --project cs203-week08-demo
```

Tab	What You See
Metrics	Training curves, overlaid runs
Media & Tables	Images, per-class tables
Runs	All configs and final metrics
System Metrics	GPU/CPU usage (auto on Apple Silicon)

Seeds control the math. TrackIO records the memory.

But your partner still can't run your code...

Level 3: Control the Machine

Docker

"Works on My Machine" — The Most Famous Lie in CS

You shared your code AND `requirements.txt`. Your friend installs everything.

```
$ python train.py
OSError: libgomp.so.1: cannot open shared object file
```

What happened? Your code depends on a system library on your Mac that doesn't exist on their Windows laptop.

Virtual environments isolate **Python packages**. They **don't** isolate:

- Operating system (Mac vs Windows vs Linux)
- System libraries (C compilers, CUDA)
- Python version itself

We need to ship the entire computer, not just the package list.

The OYO Room Analogy

Your Hostel Room (Without Docker)

Your ML project lives on your specific laptop — your Python version, your libraries, your OS quirks. If a friend wants to run your code, they have to recreate your entire setup. Usually, it crashes.

The OYO Room (With Docker)

You write a blueprint: *"I need Python 3.10, scikit-learn, my app.py, and my model."*
Your friend runs one command → an identical, fully-furnished room appears on their laptop. App runs perfectly. When they're done, the room vanishes. No mess left behind.

Without Docker	With Docker
45 mins of `pip install` errors	`docker run spam-app` (2 min)
"Which sklearn version?"	Frozen inside the container
"Are you on Windows?"	Doesn't matter
Updating a package breaks 3 projects	Completely isolated

Why Docker? Four Scenarios

1. The Assignment Defense

You submit your ML assignment. TA runs it on Windows → crash. With Docker: TA runs `docker run your-project` → guaranteed identical results.

2. The Dependency Minefield

You want to try a cool GitHub repo requiring Python 3.7 + old PyTorch. Installing it will break your other projects. Docker: isolated bubble, no damage.

3. The Group Project Peacemaker

4 teammates: Windows, Mac Intel, Mac Silicon, Linux. Getting a shared pipeline to work across all four? Docker gives everyone the exact same environment.

4. The Portfolio Builder

You built a Gradio demo and want to deploy it on HuggingFace Spaces or AWS. Cloud providers want a Docker image, not your laptop.

Six Concepts — That's All

Concept	What It Is	Analogy
Image	Blueprint — OS + libraries + code (read-only)	Recipe card
Container	Running instance of an image	Dish being cooked
Dockerfile	Instructions to build an image	The recipe
Docker Hub	Registry of pre-built images	GitHub for images
Volumes	Shared folder between container and your laptop	USB drive
Ports	How you access the container's web apps	Window into the kitchen

If Image is a **class**, Container is an **object** (instance).

Docker vs Virtual Machines

"Isn't Docker just a VM?"

```
Virtual Machine                    Docker Container

┌──────────────┐                   ┌──────────────┐
│  Your App    │                   │  Your App    │
│  Libraries   │                   │  Libraries   │
│  Guest OS    │  ← full OS!       │              │  ← no guest OS
│  Hypervisor  │                   │  Docker Engine│
│  Host OS     │                   │  Host OS     │
│  Hardware    │                   │  Hardware    │
└──────────────┘                   └──────────────┘

  ~2 GB, boots in minutes            ~150 MB, starts in seconds
```

	VM	Docker Container
Size	GBs (full OS)	MBs (just your app)
Startup	Minutes	Seconds
Isolation	Complete (own kernel)	Process-level (shared kernel)
Use case	Run a different OS	Package an app with dependencies

Docker Hub: GitHub for Images

`FROM python:3.10-slim` — where does this come from?

Docker Hub (hub.docker.com) — a registry of pre-built images.

```
# Pull an image without building anything
docker pull python:3.10-slim

# See what's available
docker images
```

Image	What's Inside	Size
`python:3.10`	Full Debian + Python + compilers	~1 GB
`python:3.10-slim`	Minimal Debian + Python	~150 MB
`python:3.10-alpine`	Alpine Linux + Python	~50 MB
`ubuntu:22.04`	Just Ubuntu, no Python	~77 MB
`nginx`	Web server	~140 MB

Always use `-slim` unless you need compilers. Your images will be 6x smaller.

Layer Caching: Why Second Build is Fast

Docker caches each step. If nothing changed, it reuses the cache:

```
$ docker build -t spam-app .
⇒ CACHED [1/5] FROM python:3.10-slim      # already downloaded
⇒ CACHED [2/5] WORKDIR /app               # nothing changed
⇒ CACHED [3/5] COPY requirements.txt .    # file unchanged
⇒ CACHED [4/5] RUN pip install ...        # packages cached!
⇒ [5/5] COPY app.py spam_model.pkl ./     # code changed → rebuilds
```

First build: 2 minutes (downloads everything).
Second build: 5 seconds (only re-copies your code).

Pro tip: That's why we `COPY requirements.txt` before `COPY . .` — if only your code changed, Docker skips the slow `pip install` step!

Environment Variables: Configure Without Rebuilding

Same image, different behavior — using `-e`:

```
# Default config
docker run -p 7860:7860 my-app

# Change model type without rebuilding!
docker run -p 7860:7860 -e MODEL_TYPE=svm my-app
```

In your Python code:

```
import os
model_type = os.environ.get("MODEL_TYPE", "rf")  # default = "rf"
```

In your Dockerfile (set defaults):

```
ENV MODEL_TYPE="rf"
ENV APP_TITLE="Spam Classifier"
```

This is how production apps are configured — DB URLs, API keys, feature flags.

Demo: `cd 5-environment` in the Docker demos

The Dockerfile: Line by Line

Our Gradio spam classifier needs this `Dockerfile`:

```
FROM python:3.10-slim              # 1. Start with Python + Linux

WORKDIR /app                       # 2. Create a workspace folder

COPY requirements.txt .            # 3. Bring the ingredient list
RUN pip install -r requirements.txt # 4. Install packages

COPY app.py spam_model.pkl ./      # 5. Bring our code + model

ENV GRADIO_SERVER_NAME="0.0.0.0"   # 6. Let the outside world connect

EXPOSE 7860                        # 7. Open the port

CMD ["python", "app.py"]           # 8. What runs when container starts
```

Line 6 is critical: Gradio defaults to `127.0.0.1` (only the container can see itself). `0.0.0.0` says "let the host laptop connect too."

Walkthrough: Build the Image

Step 1: Check Docker is running

```
$ docker --version
Docker version 24.0.5, build ced0996
```

If you get an error → open Docker Desktop first!

Step 2: Build the image

```
$ docker build -t spam-app .
⇒ [1/5] FROM python:3.10-slim
⇒ [2/5] WORKDIR /app
⇒ [3/5] COPY requirements.txt .
⇒ [4/5] RUN pip install -r requirements.txt
⇒ [5/5] COPY app.py spam_model.pkl ./
⇒ naming to docker.io/library/spam-app
```

Docker reads the Dockerfile top-to-bottom, downloads Python, installs packages, copies your code, and takes a **snapshot**. The `-t spam-app` gives it a name. The `.` means "Dockerfile is in this folder."

Walkthrough: Run the Container

Step 3: Launch your app!

```
$ docker run -p 7860:7860 spam-app
Running on local URL: http://0.0.0.0:7860
```

Open `localhost:7860` in your browser — your ML model is live!

The `-p 7860:7860` is a **port tunnel** connecting your laptop to the container:

```
Your laptop                    Docker container
┌──────────────┐                ┌──────────────┐
│              │   port 7860    │              │
│  Browser     ──────────────►  │  Gradio      │
│              │                │  server      │
└──────────────┘                └──────────────┘
```

Containers are **headless** — no screen, no browser. You look through the port window.

Walkthrough: Background & Stop

Step 4: Run in the background

```
$ docker run -d -p 7860:7860 spam-app
a1b2c3d4e5f6...
```

`-d` = "detached" — runs silently, frees your terminal. The long string is the container ID.

Step 5: See what's running

```
$ docker ps
CONTAINER ID   IMAGE      STATUS         PORTS
a1b2c3d4e5f6   spam-app   Up 2 minutes   0.0.0.0:7860→7860/tcp
```

Step 6: Stop it

```
$ docker stop a1b    # just the first 3 characters of the ID
```

Experiment 1: The Amnesia Test

Try this: Run the container. Classify some messages. Stop the container. Start a **new** container.

Question: Where did the logs/data go?

Answer: Gone. Containers have amnesia! When they die, any new files created inside them vanish. They reset to the original image snapshot.

This is **by design** — containers are disposable. But if you need to save data...

Experiment 2: The Code Change

Try this: Leave the container running. Edit `app.py` on your laptop — change the title to "Super Spam Detector v2". Refresh the browser.

What happened? Nothing changed!

The container runs a **copy** of the code from when you ran `docker build`. It doesn't watch your laptop files.

Rule: Change code → must rebuild:

```
docker build -t spam-app .     # rebuild
docker run -p 7860:7860 spam-app   # run new version
```

Experiment 3: Volumes (The USB Drive)

The fix for both experiments: Mount a **volume**.

```
docker run -v $(pwd):/app -p 7860:7860 spam-app
```

`-v $(pwd):/app` creates a synchronized tunnel between your laptop's current folder and `/app` inside the container.

Now:

- Edit `app.py` on your laptop → container sees it instantly
- Container writes a file → it appears on your laptop

Think of it as plugging a **USB drive** into the container.

Experiment 4: Hack Into the Matrix

While the container is running, open another terminal:

```
$ docker exec -it a1b bash
root@a1b2c3d4:/app#
```

You just **SSH'd into the container!** Try:

```
root@a1b2c3d4:/app# ls
app.py  requirements.txt  spam_model.pkl

root@a1b2c3d4:/app# python --version
Python 3.10.14

root@a1b2c3d4:/app# exit
```

This is a completely isolated Linux environment — separate from your Mac/Windows.

Common Docker Errors

1. `docker: command not found`
→ Docker Desktop isn't running. Open the app first!

2. `port is already allocated`
→ Something else is using port 7860. Run `docker ps`, find it, `docker stop` it.

3. `COPY failed: file not found`
→ The file isn't in the same folder as your Dockerfile. Check with `ls`.

4. Image is 3GB!
→ You used `FROM python:3.10` (full Ubuntu, ~1GB). Use `FROM python:3.10-slim` (~150MB).

```
# Clean up old images eating your disk
docker system prune
```

Think About This...

Question 1:

"If we already set seeds, why might results still differ on my Mac vs the TA's Windows?"

→ Seeds control algorithmic randomness. But different OS have different math libraries (BLAS/LAPACK), and different sklearn versions may split data differently. Docker locks the OS + versions.

Question 2:

"We have `requirements.txt` and `venv`. Why do we still need Docker?"

→ `venv` isolates Python packages. Docker isolates **everything** — OS, system libraries, Python version, CUDA drivers. `requirements.txt` + `venv` is Level 3; Docker is Level 4.

The Reproducibility Stack

Level	Tool	What It Controls
The Math	`random_state=42`	Algorithmic randomness
The Memory	TrackIO	What you tried & what worked
The Packages	`venv` + `requirements.txt`	Library versions
The Machine	Docker	OS + system libs + everything

```
Seeds + TrackIO + Venv + Docker = Time Capsule
```

You don't always need all layers. For course projects: seeds + TrackIO + venv is enough. Add Docker when shipping to production or sharing across OS.

Key Takeaways

1. **Set seeds everywhere** — `random_state=42` in every sklearn call
2. **Track experiments** — `trackio.init()` / `log()` / `finish()` → never lose a good run
3. **Pin dependencies** — `pip freeze > requirements.txt` with exact versions
4. **Dockerize for sharing** — Dockerfile + `docker build` + `docker run` = runs anywhere
5. **Volumes for persistence** — containers have amnesia; use `-v` to save data
6. **Reproducibility is a gift to your future self** — if no one else can run it, it might as well not exist